COME TO CODE 2026 · PIGNOLA

Dovremmo ancora insegnare a programmare nell'era degli LLM?

Giuseppe Mastrandrea

Tre mestieri, un problema solo

Sviluppatore. Il passato.

Insegnante di informatica. Il presente.

Formatore upskill e reskill per adulti. Il presente, quello scomodo.

Trent'anni di consigli

1995 *Everybody should learn how to program a computer,* **because it teaches you how to think**.

STEVE JOBS

2013 **Don't just play on your phone — program it.**

BARACK OBAMA · LANCIO DELL'HOUR OF CODE

2024 *It's almost exactly the opposite. It is our job to create computing technology such that* **nobody has to program**.

JENSEN HUANG · WORLD GOVERNMENTS SUMMIT

2025 *The bar to coding is* **now lower than it ever has been**.

ANDREW NG · SNOWFLAKE BUILD

2025 *In three to six months, AI is writing* **90% of the code**.

DARIO AMODEI · COUNCIL ON FOREIGN RELATIONS

2025 *I pretty much* **disagree with almost everything** *he says.*

JENSEN HUANG, SU AMODEI · VIVATECH

2026 *Le aziende che tra tre-cinque anni avranno più successo sono quelle che* **hanno raddoppiato sulle assunzioni entry-level**.

NICKLE LAMOREAUX, CHRO IBM

Tutto e il contrario di tutto.

Oh shit, here we go again

QUELLO CHE POTREMMO FARE

Aumentare le persone

Far crescere i junior più in fretta

`Claude.ai — la chat`

49% automazione

QUELLO CHE FACCIAMO

Sostituirle

Non assumerli più

`Claude Code — l'agente`

79% automazione

APERTURA Anthropic, *AI's Impact on Software Development* · 28 apr 2025 · 500.000 conversazioni

PARTE 1

Cosa dicono i dati

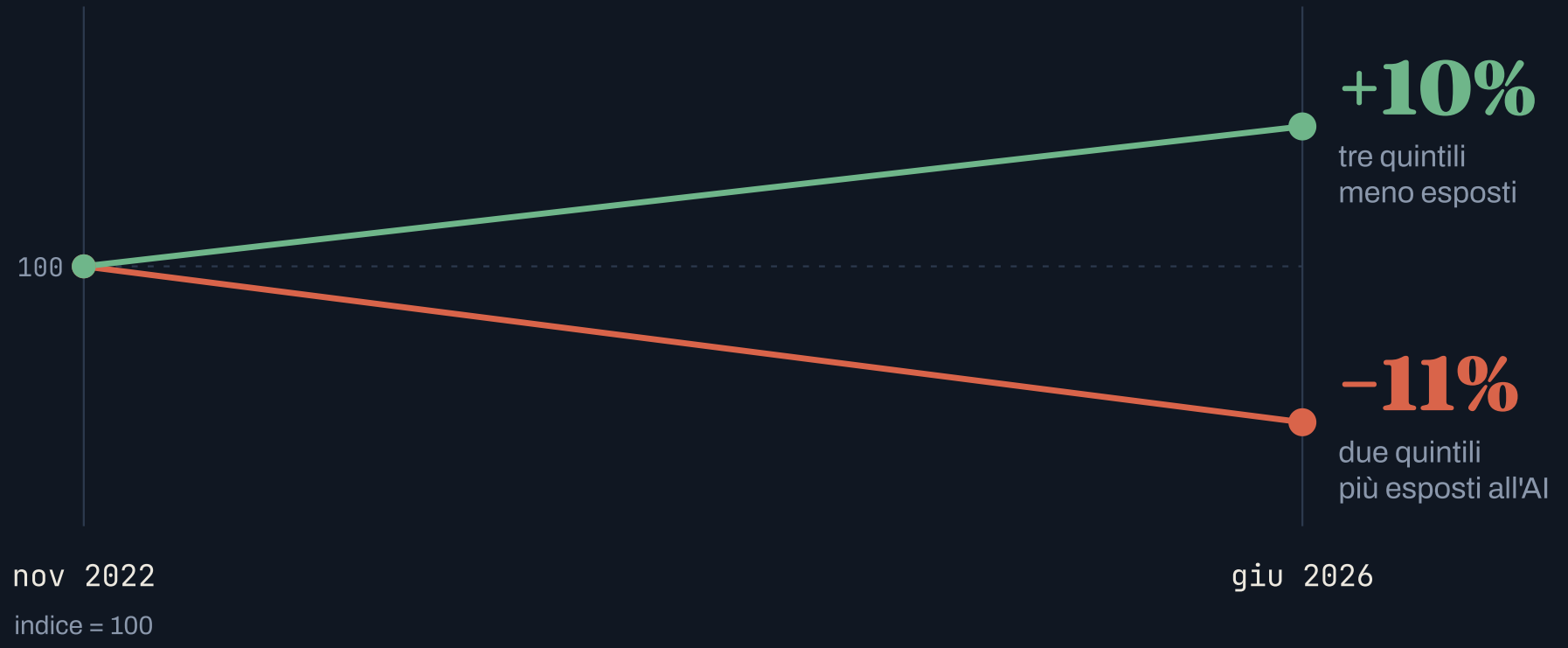
LO STUDIO CHE CONTA

Canaries in the Coal Mine?

Brynjolfsson · Chandar · Chen — Stanford Digital Economy Lab

DATI DI BUSTA PAGA ADP · MILIONI DI LAVORATORI USA · FINO A GIUGNO 2026
Non un sondaggio. Non annunci di lavoro. Record amministrativi.

Occupazione 22–25 anni



Due estremi misurati, non una serie storica: il paper riporta i livelli all'inizio e alla fine del periodo.

FATTO N. 4

...principalmente attraverso la **riduzione delle assunzioni** *di giovani, non attraverso un aumento dei licenziamenti.*

Canaries in the Coal Mine?, agosto 2026

I LIMITI, DETTI DA ME PRIMA CHE ME LI DICIATE VOI

We interpret these facts as **early, descriptive indicators** *— canaries in the coal mine —* **rather than causal estimates**.

Dall'abstract del paper

Divergenze già **prima di ChatGPT**, attorno al COVID

L'effetto **si attenua** controllando per il titolo di studio

Il campione ADP è **più marcato** dei benchmark nazionali

Il fatto che nessuno cita

DOVE L'AI SOSTITUISCE	DOVE L'AI COMPLEMENTA
Occupazione giovanile **in calo**	Occupazione **piatta o in crescita**, soprattutto fra gli esperti

CONOSCENZA CODIFICATA
Libri, tutorial, Stack Overflow.
Sostituibile.

CONOSCENZA TACITA
Esperienza, giudizio.
Resiste.

E in Italia?

50,9%

dei **laureati** che le imprese cercano è di **difficile reperimento** — 57,3% per i diplomati ITS.
Su 670.000 laureati e 120.000 ITS richiesti.

76%

degli annunci per profili ad **alta qualificazione** richiede **competenze AI**.
E il 78% delle organizzazioni fatica ad assumere: AI, dati, sicurezza.

Excelsior–Unioncamere, dati 2025 · Osservatori Politecnico di Milano

I ragazzi l'hanno già capito

70%

dei laureandi in informatica è **pessimista** sulla propria carriera.
Il 64% dei pessimisti indica la GenAI.

42%

la usa **ogni giorno**.
Media degli altri laureandi: 24%.

> ISCRIZIONI A INFORMATICA, PRIMAVERA 2026 · -8,4% ATENEI QUADRIENNALI · -11,2% BIENNALI
> Iscrizioni universitarie totali nello stesso periodo: **+1,3%**. Persistenza al secondo semestre: la più bassa fra i dieci corsi principali.

Handshake, Class of 2026 · National Student Clearinghouse, 11 ago 2026

FEBBRAIO 2026 · IL CONTRO-TREND

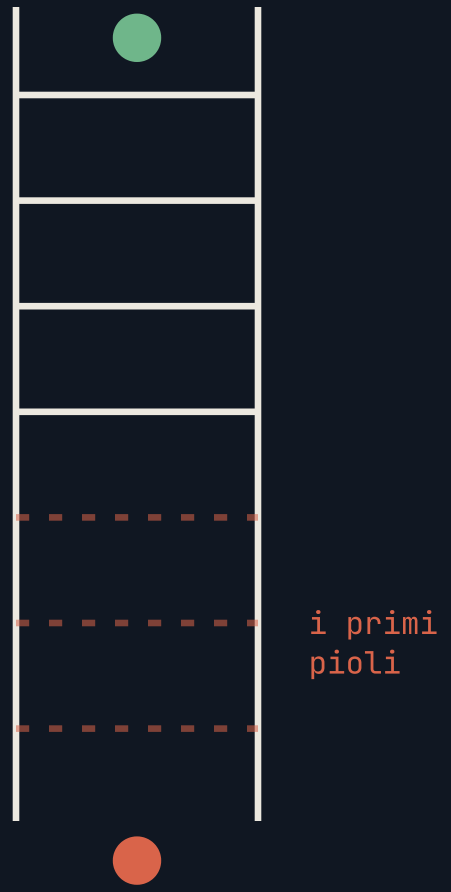
IBM triplica le assunzioni entry-level

Le aziende che tra tre-cinque anni avranno più successo sono quelle che **hanno raddoppiato sulle assunzioni entry-level** *in questo periodo.*

Nickle LaMoreaux, CHRO IBM · 12 febbraio 2026

> NON HANNO SOLO ASSUNTO DI PIÙ. HANNO RISCRITTO IL RUOLO.
> Meno coding di routine → più contatto col cliente e **validazione degli output dell'AI**.
> Il senior fa *evaluation, pattern recognition*, e insegna ai giovani a esercitare il giudizio.

PARTE 1 · I DATI Annuncio del 12 feb 2026 · l'azienda non ha diffuso cifre assolute



Nessuno ha tolto la scala.
Hanno tolto i primi pioli.

PARTE 2

Il codice è gratis. E allora?

~50%

del codice generato **non supera i test di sicurezza**.

100+ MODELLI · 80 TASK STANDARDIZZATI · 4 LINGUAGGI
Rimisurato a ogni ciclo. Chiunque può rifarlo.

Java	72%	C#	45%
JavaScript	43%	Python	38%

XSS: i modelli falliscono nell'**86%** dei casi rilevanti.

Invariato da un anno e mezzo.

Veracode GenAI Code Security Report 2025 e aggiornamento primavera 2026

36 milioni di nuovi sviluppatori su GitHub in un anno. Quasi l'**80%** usa Copilot **entro la prima settimana**.

Circa la metà del codice generato non supera i test di sicurezza — e non migliora da un anno e mezzo.

Gli sviluppatori **esperti** sono quelli che se ne fidano **meno**: diffida il **46%**, si fida il **33%**.

Stiamo formando una generazione che si fida di più proprio dello strumento di cui gli esperti si fidano di meno — senza darle gli strumenti per verificarlo.

GitHub Octoverse 2025 · Veracode 2025 · Stack Overflow Developer Survey 2025

PARTE 3

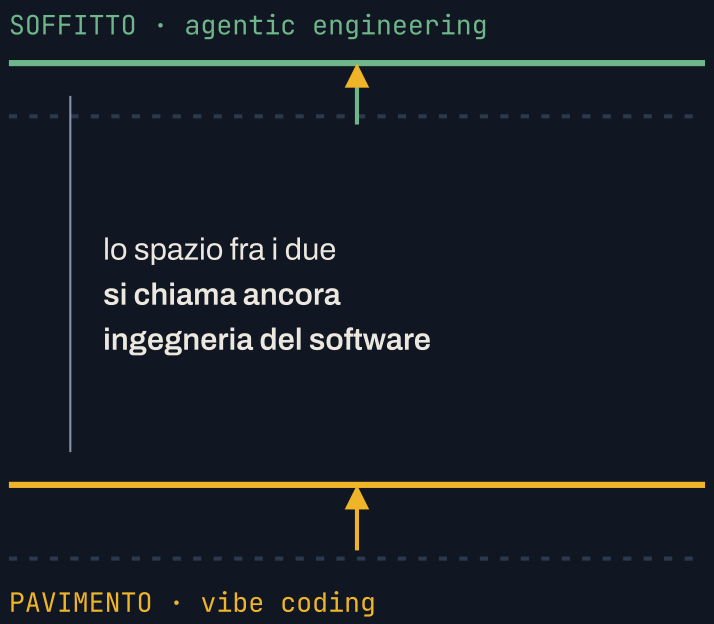
Dove va a finire il mestiere

Software 3.0

Software 1.0	scrivi **codice**
Software 2.0	alleni **pesi**
Software 3.0	scrivi **contesto** prompt, esempi, tool, memoria, istruzioni

Programming shifts from writing code to **writing context**.

Andrej Karpathy · Sequoia Ascent, maggio 2026



Pavimento e soffitto

Vibe coding alza il pavimento.
Chiunque può costruire software. Basta descrivere.

Agentic engineering alza il soffitto.
Specifiche, review, test, permessi, confini.

Puoi delegare la memorizzazione delle API. Le decisioni sulla **forma del sistema** *— identità, confini di sicurezza, integrità dei dati — restano umane.*

Andrej Karpathy · 2026

Il programma non è il codice: è la **teoria** *che i programmatori hanno in testa su come il codice rispecchia il mondo. Il codice senza la teoria è morto.*

Peter Naur · Programming as Theory Building · 1985

PARTE 4

Cosa resta da insegnare

Cosa sta comprando davvero il mercato

76%	degli annunci per profili ad alta qualificazione richiede **competenze AI**
78%	delle organizzazioni italiane **fatica ad assumere** — AI, big data, cybersecurity
50,9%	dei laureati cercati è **introvabile**

Bloom, rovesciato

PRIMA	ADESSO
Creare era la cima difficile, costoso *Ricordare* e *capire* erano la base	**Creare è gratis** **Valutare e analizzare** sono il collo di bottiglia

La lista

01 **Leggere** codice che non hai scritto

02 **Specificare** senza ambiguità

03 **Decomporre** un problema

04 **Debuggare** ciò che non hai scritto

05 **Valutare** un output sicuro di sé anche quando sbaglia

06 **Sapere quando fermarti** e non delegare

PARTE 5

L'AI in aula: sì o no?

52 SVILUPPATORI · LIBRERIA NUOVA · METÀ CON AI,
METÀ A MANO · QUIZ SUBITO DOPO

50%

CON L'AI

67%

A MANO

> 17 PUNTI PERCENTUALI · «QUASI DUE VOTI DI DIFFERENZA» · COHEN'S d = 0,738 · p = 0,01
> Il gap più grande: **sulle domande di debugging**. La velocità: +2 minuti con l'AI, non significativo.

Anthropic, *How AI assistance impacts the formation of coding skills* · gen 2026

Sei modi di usarla. Due esiti.

SOTTO IL **40%**

Delega totale
«fammelo tu»

Reliance progressiva
inizi chiedendo, finisci delegando

Debug iterativo
«aggiusta» invece di «spiega»

SOPRA IL **65%**

Genera, poi chiedi spiegazioni

Codice e spiegazione insieme

Conceptual inquiry
solo domande concettuali, errori risolti da solo

Il gruppo «conceptual inquiry» era anche il secondo più veloce in assoluto.

Anthropic RCT, gen 2026 · pattern d'uso correlati al punteggio del quiz

~1.000 STUDENTI · TRE GRUPPI · PNAS 2025

	DURANTE LA PRATICA	ALL'ESAME SENZA AI
GPT Base ChatGPT normale	**+48%**	**−17%** rispetto a chi non l'ha mai avuto
GPT Tutor stesso modello, dà hint non risposte	**+127%**	**nessun danno**

Bastani et al., *Generative AI without guardrails can harm learning*, PNAS 2025

STUDIO QUALITATIVO · 21 STUDENTI · OSSERVAZIONE, INTERVISTE, EYE-TRACKING

The Widening Gap

Chi **accelerava** usava la GenAI per scrivere codice che sapeva già di voler scrivere, e ignorava i suggerimenti sbagliati.

Chi **arrancava** ha visto le proprie difficoltà metacognitive **aggravarsi**, e ne sono emerse di nuove.

Illusione di competenza

Prather et al., ICER 2024 — *«ended with an illusion of competence»*

If letting AI in makes the task feel effortless, **it is in the wrong place.**

The Effortless Trap · Brcic & Frljic, giugno 2026

«Lo sforzo cognitivo — e persino il **restare dolorosamente bloccati** — è probabilmente importante per la padronanza.» — Anthropic, 2026

PARTE 6

Cinque esercizi che nel 2019 erano stupidi. Nel 2026 sono il programma.

ESERCIZIO 1 · PROMPT PROBLEM — allena: specificare

```
[1, 2, 2, 3, 3, 3]   →   [1, 2, 3]
[5, 5]               →   [5]
["b", "a", "b"]      →   ???
[]                   →   ???
[3, 1, 3]            →   ???
```

Scrivete il prompt. Un tentativo solo. Deve passare tutti i test.

ESERCIZIO 2 · EXPLAIN IN PLAIN ENGLISH — allena: comprendere

```
def h(xs):
    a, b = 0, 0
    for x in xs:
        a = max(a + x, x)
        b = max(b, a)
    return b
```

In UNA frase: cosa fa? Non riga per riga — cosa *fa*.

ESERCIZIO 3 · TROVA LA VULNERABILITÀ — allena:
valutare criticamente

```
@app.get("/cerca")
def cerca():
    q = request.args.get("q", "")
    log.info(f"ricerca utente: {q}")

    risultati = cerca_articoli(q)
    righe = "".join(
        f"<li><a href='/articoli/{a['id']}'>{a['titolo']}</a> — {a['autore']}</li>"
        for a in risultati
    )
    html = f"""
    <html><body>
      <h1>Risultati per: {q}</h1>
      <p>{len(risultati)} articoli trovati</p>
      <ul>{righe}</ul>
    </body></html>
    """
```

```
$ pytest -q
...                  [100%]
3 passed in 3.34s
```

I test sono verdi. È in produzione.
Chi la vede?

NON UNA. TRE.

01 **XSS riflessa** su `{q}` nell'h1

02 **XSS stored** su `{a['titolo']}` — se i titoli arrivano dal DB. Questa non la vede quasi nessuno.

03 **Log injection**: con un `%0A` forgi una riga di log falsa

```
2026-09-18 16:57:54 INFO ricerca utente: python
2026-09-26 10:00:00 INFO utente admin: login riuscito da 10.0.0.1   ← FALSA
```

```
def calcola_totale(ordine):
    # 1. Subtotale degli articoli
    subtotale = sum(a["prezzo"] * a["quantita"] for a in ordine["articoli"])

    # 2. Promozione percentuale (es. saldi -20%)
    if ordine.get("promo_pct"):
        subtotale = subtotale * (1 - ordine["promo_pct"] / 100)

    # 3. Punti fedelta': 1 punto = 1 centesimo di sconto
    punti_usati = min(ordine.get("punti", 0), int(subtotale / VALORE_PUNTO))
    subtotale = subtotale - punti_usati * VALORE_PUNTO

    # 4. Coupon a valore fisso
    if ordine.get("coupon"):
        subtotale = max(0.0, subtotale - ordine["coupon"])

    # 5. Spedizione: gratis sopra la soglia
    spedizione = 0.0 if subtotale ⩾ SOGLIA_SPEDIZIONE_GRATIS else COSTO_SPEDIZIONE
```

Il marketing chiede: la soglia spedizione deve valere sul carrello *prima* degli sconti, e i punti non devono maturare sull'IVA. Dove mettete le mani?

CARRELLO: 2 × 32,50 € · PROMO -20% · NESSUN COUPON

Senza usare i punti	63,44 €
Usando 300 punti (= 3,00 € di sconto)	68,20 €

Spendere 3,00 € di punti fa aumentare il totale di 4,76 €.

I punti fanno scendere il subtotale da 52,00 a 49,00 — sotto la soglia dei 50 € — e scatta la spedizione a 6,90 più IVA.

Peter Naur, *Programming as Theory Building*, 1985

ESERCIZIO 5 · CODE REVIEW AL CONTRARIO — allena:
decidere e motivare

Issue: «Il totale del carrello a volte è sbagliato di un centesimo.» Due PR.
Entrambe verdi. Entrambe generate.

PR #1 +2 -1

```
def totale(prezzi, sconto_pct):
    t = sum(prezzi)
    return round(
        t - t * sconto_pct / 100, 2)
```

PR #2 +9 -3

```
t = sum(Decimal(str(p)) for p in prezzi)
sconto = (t * Decimal(str(sconto_pct))
    / Decimal(100)).quantize(
    Decimal("0.01"), ROUND_HALF_UP)
return (t - sconto).quantize(...)
```

Quale approvate? E cosa scrivete nel commento?

CARRELLO	SCONTO	PR #1	PR #2
[28.20, 55.44, 22.01]	10%	95,09	95,08
[41.56, 54.89]	50%	48,23	48,22
[12.02, 36.51, 20.98]	50%	34,76	34,75

```
>>> round(2.675, 2)
2.67        # non 2.68: banker's rounding, non half-up
```

Cosa hanno in comune

1 · Prompt Problem	**Specificare**
2 · Explain in Plain English	**Comprendere**
3 · Trova la vulnerabilità	**Valutare criticamente**
4 · Il test di Naur	**Costruire la teoria**
5 · Code review al contrario	**Decidere e motivare**

In nessuno dei cinque il compito è **scrivere codice**.

In tutti e cinque, non li sai fare se non hai mai scritto codice.

DOVREMMO ANCORA INSEGNARE A PROGRAMMARE NELL'ERA DEGLI LLM?

Sì.
Più di prima.

Ma non per **produrre** codice — quello è gratis. Per costruire la **teoria** che rende il codice vivo, e la capacità di **giudicare** una macchina che sbaglia con sicurezza.

Tre cose

01 **Automazione o aumento**: la stessa tecnologia, due esiti opposti. Chi insegna decide quale.

02 **La facilità è un segnale d'allarme**, non un obiettivo. Per gli studenti e per voi.

03 **I pioli bassi non si automatizzano impunemente.** Qualcuno deve restare a costruirli.

Grazie

Giuseppe Mastrandrea

Tutte le fonti, con i numeri verificati uno per uno — compresi gli studi che mi danno torto.

`Train you must. Not with a saber — with knowledge.`